Let's Refine

SQL →

→create table SCHOOL(ID integer primary key, NAME varchar(10) not null, STRENGTH integer));

→create table SCHOOL(ID integer, NAME varchar(10), STRENGTH integer, primary key(ID));

→create table SCHOOL(NAME varchar(10) , STRENGTH integer));

alter table SCHOOL add ID integer primary key;

create table SCHOOL(ID integer , NAME varchar(10) , STRENGTH integer));

alter table SCHOOL add primary key(ID);

alter table SCHOOL modify ID integer primary key;

→alter table STUDENT drop primary key;

ORDER BY→

```
SELECT * FROM EMP
WHERE DEPTNO=10
ORDER BY NAME, JOB;
```

Some SQL commands

1) To create a database

CREATE DATABASE DATABASENAME;

e.g. CREATE DATABASE ABC;

---

2) To find out all databases in SQL environment

SHOW DATABASES;

---

3) To open a Database

USE DATABASENAME;

e.g. USE ABC;

---

4) To delete a Database

DROP DATABASE DATABASENAME;

e.g. DROP DATABASE ABC;

---

5) To find out all tables in a Database

SHOW TABLES;

---

6) To Display Structure of a Table

DESC TABLENAME;

e.g. DESC EMP;

Python was developed by **Guido Van Rossum** in 1990's
Python has two part—

1. **Shell mode** (interactive mode or command mode) – where we can apply single command at a time.
2. **Script mode** (programming mode) – where we write our program

Adding element in List→ L.append(10)
Adding element in tuple→T=T+(10,)

File opening→ f=open('abc.txt', 'w') with open('abc.txt', 'w') as f

==========

EXCEPTION HANDLING

```
try:
        f=open('abc.txt', 'w')
except FileNotFoundError:
        print('File is missing')
except PermissionError:
        print('you are not allowed to use this file')
except IOError:
        print('File not readable or writeable')
else:
        s=f.read()
        print(s)
finally:
        print('file operation ends here')
```

==========

```
L=[2,3,4,5,6]
try:

   n=int(input('Enter index number'))
   if L[n]%2!=0:
            raise Exception
except :
        print('Odd number')
```

Note→ try block must be followed by except block or at least finally block

==========

Tokens in Python – Smallest individual unit of Python program . Tokens are also called lexical units. Tokens are of 5 types – Keywords, Identifiers, Literals, Punctuators and Operators

1. **Keywords** are reserve words of python with special meaning –
   like if, for, or, break, while, return etc.
   True ,False and None are only keywords which starts with capital or upper case letter, rest of all are all in lower case.
2. **Identifiers** are name given to variable, function, list, string, dictionary etc.
   a. Rules of naming identifier—
      i. Name can contain alphabet, digit or underscore only
      ii. Name can contain digit but can't start with digit
      iii. Name can't be a keyword
      iv. Upper case name and lower case name are different
      - e.g. Rollno ok Roll no wrong Rollno. Wrong
      - Roll-no wrong Roll_no ok Rollno_ ok
      - Roll1no ok Rollno1 ok 1Rollno wrong
      - True wrong (variable name can't be a keyword)
      - true ok (as it is not a keyword)
      - None wrong none ok
      - Roll$no wrong Roll*no wrong
3. **Literals** are values given to variables
   Like A=10

S='Raj'

Then 10 and 'Raj' are literals

a. Types of literals
   i. Numeric
      i. Integer → like 3, 100, 507 etc.
      ii. Boolean → Like True, False
      iii. Special literal **complex to store complex number real+imag j**
      e.g. a=5+2.5j (where a.real=5 and a.imag=2.5)
   ii. String
      → Like 'hello', "school", '''Sunil''' (single, double and triple quotes can be used)
   iii. None (means absence of value)

4. **Punctuators** are punctuations used in python
   Like ' " : ; , # ( ) { } [ ] etc.
5. **Operators** are symbols which define some operation

Types of operator according to **number** of operands→

i) **Unary operator** → Which works on single operand

Like A=10

print (-A)

Output -10 Here – is working on single operand A

ii) **Binary operator** → Which works on two operands

Like A+B A-C A*B A/B A=B A>B etc.

Types of operators according to **nature** of operator—

i. **Arithmetic operator** -- used for arithmetic calculations →Like + - * / // % ** etc.

Some examples :

5/2=2.5 9/3=3.0 5//2=2 5.0//2= 2.0

5//2.0=2.0 6.5//2=3.0 5%2=1 7.2%3=1.2

6%2.5=1.0 3**2=9 (math.pow (3,2)=9)

-3**2= -9 math.pow (-3,2)=9.0

2*2*3=**256**(as it tends to solve from Right and Left(firstly solve 2**3 i.e.8 then 2**8=256))

with String →

'Hello' + '!' = Hello! 'Hello' + '5' = Hello5 'Hello' + 5 = Error (string can't be add with number)

'Hello' * 4 = HelloHelloHelloHello Hello' * '4' = Error (4 must be an integer)

With List and Tuple→

L1=[10,20,30] L2=[11,22,33]

print (L1+L2) → Output [10,20,30,11,22,33] print(L1+2) → Error

print(L1+[2]) → [10,20,30,2] print(L1+['S']) → [10,20,30, 'S']

print(L1*2) →[10,20,30, 10,20,30]

T=(10,20,30)

print(T+[2]) → Error print(T+(2)) → Error

print(T+(2,)) →(10,20,30,2) a=100

print (L1+[a]) → [10,20,30,100] print (T+('a',)) → (10,20,30, 'a')

With Boolean →

True +1=2 True – 1 =0 False – 3= –3 (As value of True is 1 and value of False is 0)

- Operator Precedence → It is **order** in which operators are solved
  Firstly ** is solved, then * / // %, and then + -
  Like 6/3+3**2**2+5*2/4 = 85.5
  Like 3+9//2-2 = 5 (as // is solved before + -)
  Like 2 * 3//5 =1
  Like 3//5 * 2 = 0 (Both (* aur //) has same precedence but expression is solved from left to right
- Associativity → It is **direction** in which operators are solved.
  Like 3/2*5-2= 5.5 (Expr. is solved from Left to Right)
  But A=10 (Expr. is solved from Right to Left)
  Likewise --
  A=5
  B=10

A=B
print (A, B)
Output: 10 10 (As in A=B equation, B is assigned to A as direction of solving operator is from Right to Left.

ii. **Relational operators** --Used to compare two values and result into True or False

Like > , < , >= , <= , == , != , >= , <=

2+3>4 True

(As precedence of Arithmetic operator is more than Relational operator therefore firstly 2+3 is solved)

'H' > 'h' False (As Ascii value of 'H' is lesser than 'h')

'hi' > 'hello' True (Here first letter is same so compare second letter)

L1=[2,4,6]

L2=[3,5,7]

L1>L2 False

L1=[3,5,8]

L2=[3,5,7]

L1>L2 True

iii. **Logical operators**→Used to join two conditions or for negation of conditionLike and, or, not

2>3 and 5 <7 False 2>3 or 5 <7 True not(2>3) True

Here 0 and ' ' means False, remaining all values refers to True

0 or 8	ans 8	-5 or 0	ans -5	'H' or ' '	ans H
0 and 8	ans 0	-5 and 0	ans 0	'H' and ' '	ans ' '

Special equation→ 5 or 6 ans 5 5 and 6 ans 6

In OR operator if first condition is True then second condition is not checked and if first condition is False then only second condition is checked whereas in AND operator if first condition is False then second condition is not checked and if first condition is True then only second condition is checked

Therefore

5>2 or 7==3/0 ans is True even 3/0 is an error (Division by Zero)

Now Chained comparison with 'and' operator

Like 11>8 <9 ans True its meaning is 11>8 and 8 < 9

Likewise 11<8 <9 ans False

iv. **Assignment operator (=)**→Used to assign value to a variable

Associativity- Right to Left as discussed above

a =b=10 means a=10 b=10

e.g. X=10

Y, Y =X+2, X+5

print (X,Y)

Output 10 15 as Y will contain latest value of X

Another e.g. x, y =5,8

x, y, x=x+2, y+3, x+8

print(x, y)

Output → 13 11

What is Compound assignment?--> Using Arithmetic operator with = sign. It is also known as Shorthand or Augmented operator

a=a+10 can be written as a+=10

A=A/B as A/=B

How to swap two values in Python?

x, y = 10, 20

x, y = y, x

print (x, y)

output: → 20 10

What is Rvalue? → which can only be specified in Right of assignment operator

e.g. A=10 Here 10 is Rvalue

We can't specify it on Left side Such as 10=A It is wrong statement

What is Lvalue? →Which can be specified in left or right of assignment operator

A=10

B=20

A=B

B=A Here A and B is Lvalue as they can be specified in either side of =

v. **Special operator** → Used in special operations → Two types

A) Identity operator (is , is not)

A=10
B=10
A is B ans. True (Because both are referring to same value in memory)
A=10
B=20
A is B ans. False
A=10
B=20
B=B-10
A is B ans. True

B) **Membership operator (in , not in)**

'S' in 'School' ans True
'S' in 'school' ans False
5 in [4,5,7,2] ans True
5 not in [4,5,7,2] ans False

vi. **Bitwise operators --** Works on bit i.e. 0 and 1

Types of bitwise operators—

- Bitwise and (&) 101 & 110 = 100
- Bitwise or (|) 101 | 110 = 111
- Bitwise not or complement (~) ~1011 = 0100
- Bitwise XOR (^) 101 ^ 111 = 10
- Right shift (>>) 1010>>1=0101
- Left shift (<<) 101<<1=1010

What is Dynamic typing?

In python we don't have to declare a variable they are declared implicitly and data type of variable is not fixed, same variable which is of integer type become of string type if we assign string value to it. It become of float type if we assign float value in it. This is known as Dynamic Typing, as shown below in example

X=10 (X is integer)
X='Hello' (Now X is a String)
X=10.5 (Now X is a float)

What is Type conversion? →Changing of datatype of a variable. It can be of two types

A) **Implicit conversion or Coercion** (happens automatically in Python as per behaviour of operator)

e.g. 9/3 =3.0(Here two integers are divided but / operator convert the result automatically to Float) This is done by python implicitly

B) **Explicit conversion or Casting** (Done by user for specific need)

int (9/3) = 3 (Here result of divide operation is converted to integer by programmer by applying command) It is of explicit type

What is Datatype - it defines type of data a variable can hold or store. Different Data Types are→

A) Number i) Integer ii) Float iii) Boolean
B) Sequence i) String ii) Tuple iii) List
C) Mapping i) Dictionary
D) None (Blank value)

What is Escape sequence? →Special combination of characters with \ sign

Like → \n for newline (\r also) \t for tab \a for alarm or beep sound other are → \' \" etc.
Used for special case also such as 'Seema's pen'→Error
→It should be 'Seema\'s pen' OR "Seema's pen"
If we calculate length then→ len('Seema\'s pen') = 11 and len("Seema's pen") = 11
len("Seema's p\en") = 12

What is bool function? →It return True or False as per given value →False for 0 and True for number other than 0

Like bool (1) ans True bool (8) ans True bool (-8) ans True bool (0) ans False
bool ('e') ans True bool ('hello') ans True bool (' ') ans False bool (' ') ans True Space is also a character

How to print memory address of a variable?

By id () function Like A=10 print (id (A)) → output: 140716279260232 (any address)

Which command is used as empty statement in python?

Ans. Pass Example Program to print only those words of a string which doesn't starts with vowel

```
S= input ('Enter String')
L=S.split( )
for w in L:
    if w [0] in 'aeiouAEIOU':
        pass
    else:
        print (w)
```

More on behaviour of String→String can be enclosed in Single quotes ' 'Double quotes " " or Triple quotes ''' '''
(Three times single quotes)

String can be --

i) Single line like ' Hello Jay'

ii) Multiline like '''Hello
Jay''' Or 'Hello\
Jay' Or "Hello\
Jay"

Note it down that when we use single or double quotes for multiline string then to break the line we have to mention '\' sign

```
print (len ('Hello\
Jay'))
```

Output 8

```
print (len ('''Hello
Jay'''))
```

Output 9

Because in triple quotes \n is hidden which is considered as one character therefore length is 9

Comments in python→ Non executable statement of pyton program. Two types

A) **Single line comment** → # is used to make a single line comment. It is also of two types

i) full line →Like # s=input ('Enter string')

ii) inline →Like A=B%10 # used to calc mod

B) **Multiline comment** → ''' ''' triple quotes are used to make multiline comment (which is also known as DocString)

```
''' This is a program to calculate factorial of a number N
    which means
        product of all numbers between 1 and N '''
```

→ this is DocString

Is it possible to write two commands in a single line in python?

Yes, it is possible by using ; sign

Like → print (a) ; print (b)

Is it possible to write a long single command in two or multilines in python?

Yes, it is possible by using \ sign

Like →

```
for i in \
    range(10,20,2):
```

String Slicing

S='amazing'

-7	-6	-5	-4	-3	-2	-1
a	**m**	**a**	**z**	**i**	**n**	**g**
0	1	2	3	4	5	6

Then--

S [0:7] amazing S [0:3] ama S [2:5] azi S [-7 : -3] amaz S [0:5] amazi

S [3 : 10] zing (10 is not index but no error will generate in slicing only)

S [-4 :-9: -1] zama (-9 is not index but no error will generate in slicing only)

S [: 5] amazi (Because Default value of Starting index is 0)

S [3:7] zing S [3:] zing (Because Default value of Ending index is last index+1)

S [3:-2] zi S [:3] + S [3:] amazing S [:-5] am S [-5:] azing

S [1:6:2] mzn S [-7 :-3: 3] az S [5 :1: -2] nz S [: : -1] gnizama (Reverse of String)

```
S='NAYAN'
A=S [ : : -1]
if A==S:
```

```
    print ('Palindrome String ')
else:
    print ('Not ')
```

What is SHALLOW and TRUE copy of LIST

```
L=[1,2,3]                          L=[1,2,3]
G=L                                G=list(L)
L[1]=5                             L[1]=5
print(L)        [1,5,3]            print(L)        [1,5,3]
print(G)        [1,5,3]            print(G)        [1,2,3]
```

Here G is Shallow copy of List **Here G is True copy of List**

Connecting Sql and Python

Suppose emp table is having 10 records then you apply query→select * from emp where sal>50000;
and suppose 6 record are filtered, these 6 records are known as "Recordset"
Now if we apply command -----
1) cursor.fetchall()
print (cursor.rowcount) then output → 5
if we apply below command instead of first command -----
2) cursor.fetchmany(3)
print (cursor.rowcount) then output → 3
if we apply below command instead of first and second command -----
3) cursor.fetchone()
print (cursor.rowcount) then output → 1
But now observe carefully
if we apply below commands in a row instead of above commands -----
4) cursor.fetchone()
print (cursor.rowcount)
print (cursor.rowcount) then output 1 (fetching only first record)
2 (because now you are fetching second record of Recordset)
And if we apply one more time this command
print (cursor.rowcount) then output 3